Tastigo Projet NoSQL

Services autonomes réplicas événementiels et projection de suivi

Idris et Rayan
Dépôt github.com/idris404/Tastigo
Mini rapport de projet - novembre 2026

Objet de l'application

Tastigo est une application pédagogique de livraison de repas. Un client consulte un catalogue, valide une commande puis suit son acheminement. Le projet montre comment quatre services restent autonomes grâce à leurs propres agrégats, à deux technologies NoSQL et à des réplicas mis à jour par événements.

Le livrable correspond au niveau sans programmation complète des services : les contrats sont définis en JSON, MongoDB et Redis sont remplis et interrogeables, et un scénario automatisé démontre la création et la mise à jour des réplicas ainsi que la projection destinée au client.

Périmètre

- Restaurant gère les restaurants, les horaires et les menus.
- Commande construit le panier, fige les prix et valide la commande.
- Livraison organise l'acheminement à partir d'un snapshot de la commande.
- Suivi transforme les événements en une vue de lecture simple pour le client.

Architecture et agrégats

Chaque frontière de service suit une responsabilité métier. Un service ne lit jamais directement la base d'un autre service. Les informations nécessaires sont copiées localement par événement.

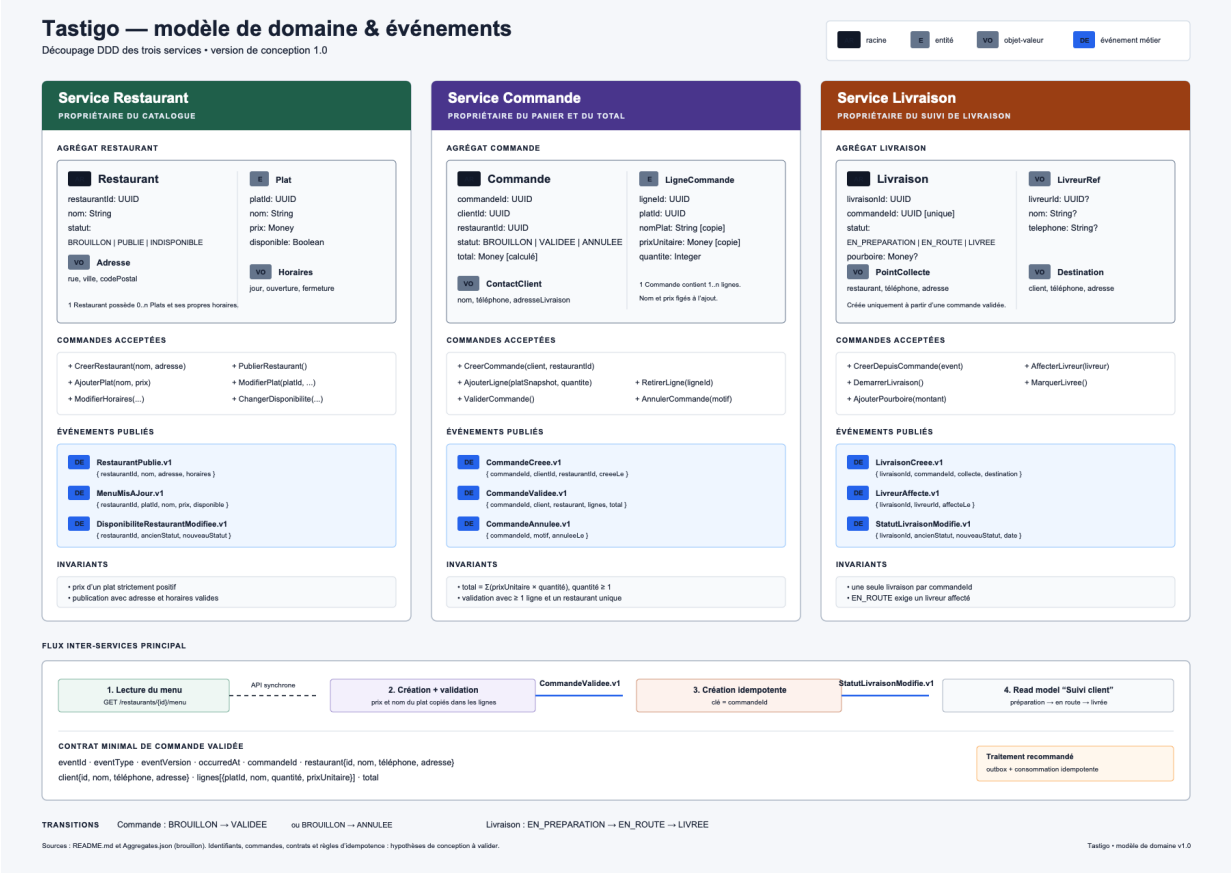


Figure 1 Agrégats réplicas et flux événementiels

Service	Agrégat propriétaire	Réplica local	Stockage
Restaurant	Restaurant et plats	Aucun	MongoDB
Commande	Commande et lignes	Catalogue Restaurant minimal	Redis DB 2
Livraison	Livraison	Snapshot de commande	Redis DB 3
Suivi	SuiviCommande	Résumé et état de livraison	Redis DB 4

Structures principales

Restaurant contient une adresse, des horaires et une liste de plats. Commande contient le client, les lignes avec nom et prix figés, le total et le statut. Livraison contient les points de collecte et de destination, le livreur et le statut. SuiviCommande contient un résumé, un libellé client et une chronologie.

Choix des bases NoSQL

MongoDB pour le catalogue Restaurant

MongoDB stocke l agrégat Restaurant sous forme de document imbriqué. Une lecture renvoie directement le restaurant, son adresse, ses horaires et ses plats. Ce choix correspond à un catalogue lu comme un ensemble et évite des jointures. Un validateur JSON Schema contrôle les champs obligatoires et le prix positif.

- Index unique sur restaurantId.
- Index composé sur le statut et la ville.
- Index textuel sur le nom des plats.
- Collection outbox pour les événements non encore publiés.

Redis pour Commande Livraison et Suivi

Redis est adapté aux accès rapides par identifiant et aux changements de statut. Les HASH stockent les champs simples, les LIST conservent les lignes et la chronologie, les SET mémorisent les événements déjà traités et les STREAM représentent les événements sortants. Les bases logiques séparent les services.

Structure Redis	Utilisation
HASH	Agrégat, réplica et projection par identifiant
LIST	Lignes de commande et chronologie de suivi
SET	Index et déduplication par eventId
STREAM	Événements produits par Commande et Livraison

Compromis

La réplication améliore l'autonomie et la disponibilité, mais la cohérence devient éventuelle. Un court délai peut exister avant la mise à jour du réplica. eventId garantit l'idempotence et aggregateVersion permet d'appliquer les modifications dans l'ordre.

Commandes événements et réplication

Commands.json définit huit demandes de traitement, notamment PublierRestaurant, ModifierPrixPlat, ValiderCommande, ModifierAdresseLivraison, AffecterLivreur et ModifierStatutLivraison. Events.json définit sept contrats avec émetteur, récepteur, donnée modifiée, ancienne valeur et nouvelle valeur.

Flux	Création du réplica	Mise à jour
Restaurant vers Commande	RestaurantPublie	MenuMisAJour
Commande vers Livraison	CommandeValidee	AdresseLivraisonModifiee
Commande et Livraison vers Suivi	CommandeValidee	LivraisonCreee et StatutLivraisonModifie

Scénario démontré

- MongoDB crée deux restaurants et place RestaurantPublie puis MenuMisAJour dans l outbox.
- Le consommateur crée le catalogue répliqué dans Redis DB 2 puis fait passer le prix de 1 250 à 1 290 centimes.
- CommandeValidee crée un snapshot autonome dans Livraison en Redis DB 3.
- AdresseLivraisonModifiee remplace l adresse du snapshot avant le départ.
- Les événements CommandeValidee, LivraisonCreee et StatutLivraisonModifie construisent SuiviCommande en Redis DB 4.

Projection SuiviCommande

La projection transforme les données sources au lieu de les recopier telles quelles. Elle conserve le nom du restaurant, le nombre d articles, le total, le statut lisible Votre commande est en route et une chronologie ordonnée. Le front end peut ainsi répondre à la question où en est ma commande avec une seule lecture.

Exécution et vérification

Le projet nécessite Docker Desktop, jq et Python 3. Depuis la racine du dépôt :

./scripts/demo.sh Démarre les bases, remplit les données, traite les événements et affiche les réplicas.

./mongodb/run-queries.sh Exécute les recherches et agrégations MongoDB de démonstration.

./scripts/validate.sh Valide les JSON, Docker, les deux bases, les réplicas et la projection.

Requêtes vérifiées

- Lister les restaurants publiés à Paris.
- Afficher les plats disponibles et leur prix.
- Lister les événements non publiés de l outbox.
- Lire le catalogue répliqué, le snapshot de commande et le suivi client.

Résultat

Le contrôle final retourne VALIDATION_OK avec quatre services, huit commandes, sept événements et deux technologies NoSQL. Le prix répliqué vaut 1 290 centimes, la version source vaut 3 et aucun événement de l outbox ne reste en attente.

Limites

Le projet démontre les structures, les requêtes et les flux sans exposer d'API HTTP ni d'interface utilisateur. Dans une version de production, un broker, des transactions d'outbox complètes, l'authentification, la supervision et des tests de concurrence seraient nécessaires.